

中国科学院大学
University of Chinese Academy of Sciences

Generative Models & AI4S Applications

Lecture 8 Normalizing Flow II

Fall 2026

Liu Yunfan (刘云帆)

liuyunfan@ucas.ac.cn

Announcements

- Assignment 1 is due on **October 18th**

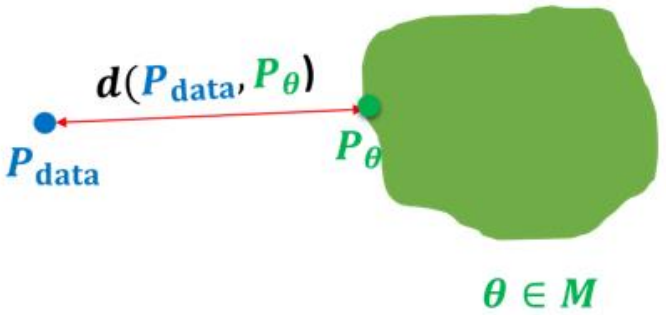
- Assignment 2 will be released after the deadline of assignment 1

Recap: Normalizing Flow Models

Images of dogs



$x_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$



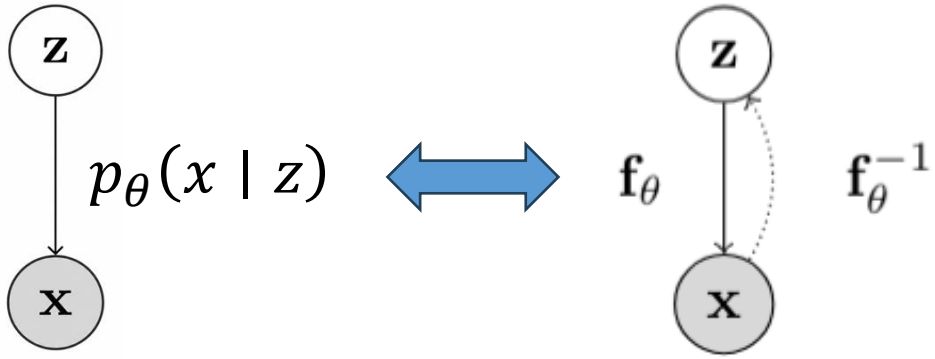
Dog Image Generator

- **Goal:** learn a probability distribution $p_\theta(x)$ over images x
  - **AR:** $p_\theta(x) = \prod_{i=1}^{n} p_\theta(x_i \mid x_{<i})$ ➔ tractable likelihood, but no latent variable
  - **VAE:** $p_\theta(x) = \int p_\theta(x, z)\, dz$ ➔ intractable marginal likelihood, do have latent variables
- **Motivation:** latent variable model + tractable likelihood? ➔ NF !

Recap: Normalizing Flow Models

- **Model Representation**
  - **Similar to VAE:** A normalizing flow model is a directed, **latent-variable model** over observed variables X and latent variables Z

Variational Autoencoder



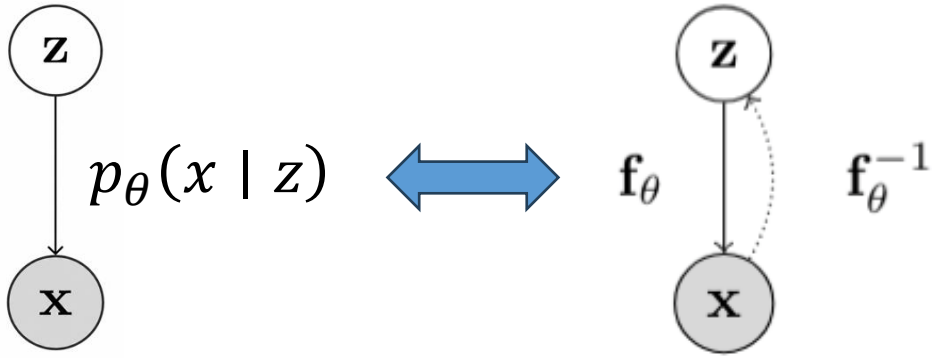
Normalizing Flow Models

  - **Unlike VAE:** The mapping between Z and X, given be $\boldsymbol{f}_\theta: \mathbb{R}^n \rightarrow \mathbb{R}^n$, is **deterministic** and **invertible**, i.e, $X = \boldsymbol{f}_\theta(Z)$ and $Z = \boldsymbol{f}_\theta^{-1}(X)$

Recap: Normalizing Flow Models

- **Model Learning**

Variational Autoencoder



Normalizing Flow Models

➢ **Goal:** Learning $p_X(x;\theta)$ with the principle of **maximum likelihood**

✓ For normalizing flow models, the marginal likelihood is

$$p_X(x;\theta) = p_Z\left(f_\theta^{-1}(x)\right)\left|det\left(\frac{\partial f_\theta^{-1}(x)}{\partial x}\right)\right|$$

Unlike VAE, the marginal likelihood can be **computed exactly**!

Recap: Normalizing Flow Models

- **Construction of the Transformation f_θ**
  - **A General Solution:** Composing a sequence of M simple invertible transformations ($x \triangleq z_M$)

$$z_M \coloneqq f_\theta^M \circ \cdots \circ f_\theta^1(z_0) = f_\theta^M\left(f_\theta^{M-1}\left(\cdots\left(f_\theta^1(z_0)\right)\right)\right) \triangleq f_\theta(z_0)$$

  - **Using Change of Variables:**

$$p_X(x;\theta) = p_Z\left(f_\theta^{-1}(x)\right)\prod_{m=1}^{M}\left|\det\left(\frac{\partial (f_\theta^m)^{-1}(z_m)}{\partial z_m}\right)\right|$$

Computational Complexity $\boldsymbol{O(n^3)}$

  - Property of transformation f_θ

① Invertible; ② Expressive; ③ Computationally Tractable

Recap: Normalizing Flow Models

- **Efficient Computation of Jacobian's Determinant**
  - **Motivation:** Design f_θ so that the Jacobian has special structure
    - ✓ Determinant of a **triangular matrix** is the **product of the diagonal entries**, and thus an $\boldsymbol{O(n)}$ operation

$$\mathbf{x} = (x_1, \cdots, x_n) = \mathbf{f}(\mathbf{z}) = (f_1(\mathbf{z}), \cdots, f_n(\mathbf{z}))$$

$$J = \frac{\partial \mathbf{f}}{\partial \mathbf{z}} = \begin{pmatrix} \frac{\partial f_1}{\partial z_1} & \cdots & \frac{\partial f_1}{\partial z_n} \\ \cdots & \cdots & \cdots \\ \frac{\partial f_n}{\partial z_1} & \cdots & \frac{\partial f_n}{\partial z_n} \end{pmatrix} \xrightarrow{x_i = f_i(\mathbf{z})} J = \frac{\partial \mathbf{f}}{\partial \mathbf{z}} = \begin{pmatrix} \frac{\partial f_1}{\partial z_1} & \cdots & 0 \\ \cdots & \cdots & \cdots \\ \frac{\partial f_n}{\partial z_1} & \cdots & \frac{\partial f_n}{\partial z_n} \end{pmatrix}$$

$\boldsymbol{O(n^3)}$

Only depends on $\boldsymbol{z_{\leq i}}$

$\boldsymbol{O(n)}$

  - **Goal:** Design f_θ with triangular Jacobian matrix ➔ **Linear complexity**



Lecture Outline

- **Flow Models with Triangular Jacobians**

- **Autoregressive Flow Models**
  - ✓ **MAF Models**
  - ✓ **IAF Models**

- **Probability Distillation and Parallel WaveNet**



Lecture Outline

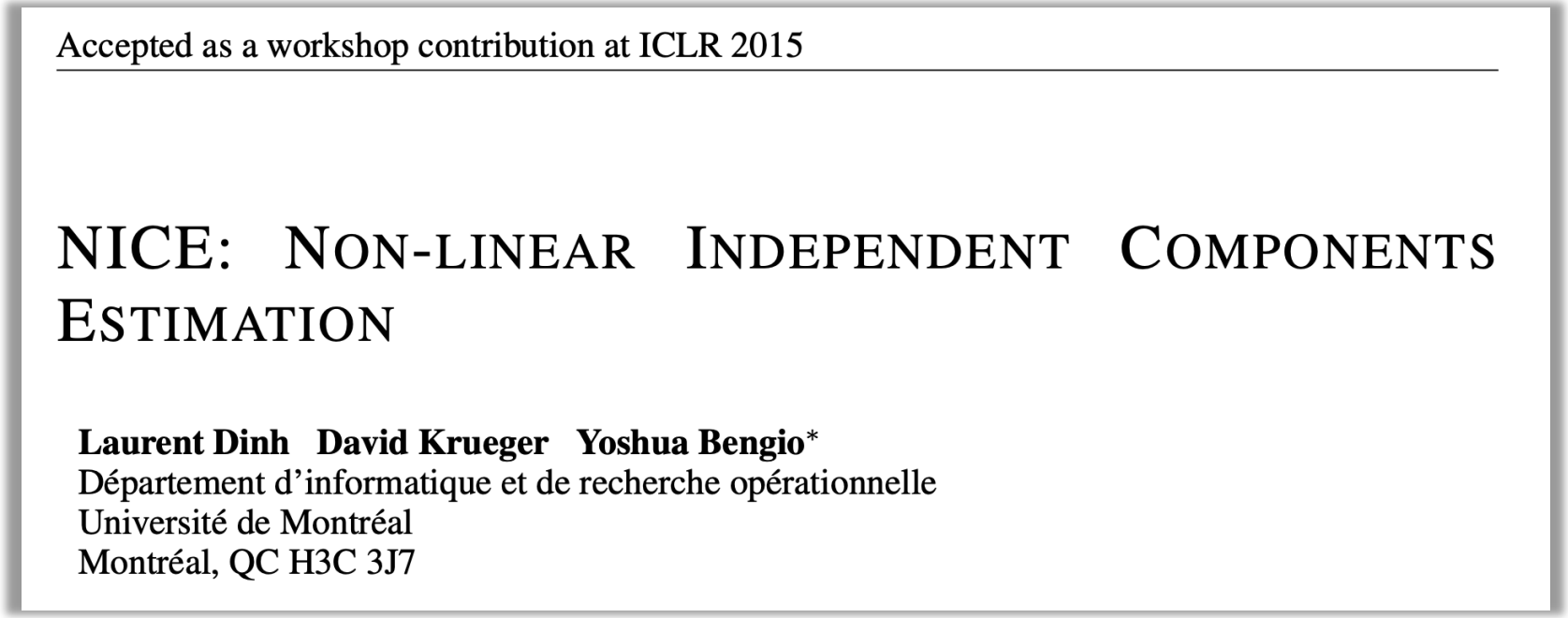
- **Flow Models with Triangular Jacobians**

- **Autoregressive Flow Models**
  - ✓ **MAF Models**
  - ✓ **IAF Models**

- **Probability Distillation and Parallel WaveNet**

Flow Models with Triangular Jacobians

- **Non-linear Independent Components Estimation (NICE)**



Accepted as a workshop contribution at ICLR 2015

NICE: NON-LINEAR INDEPENDENT COMPONENTS ESTIMATION

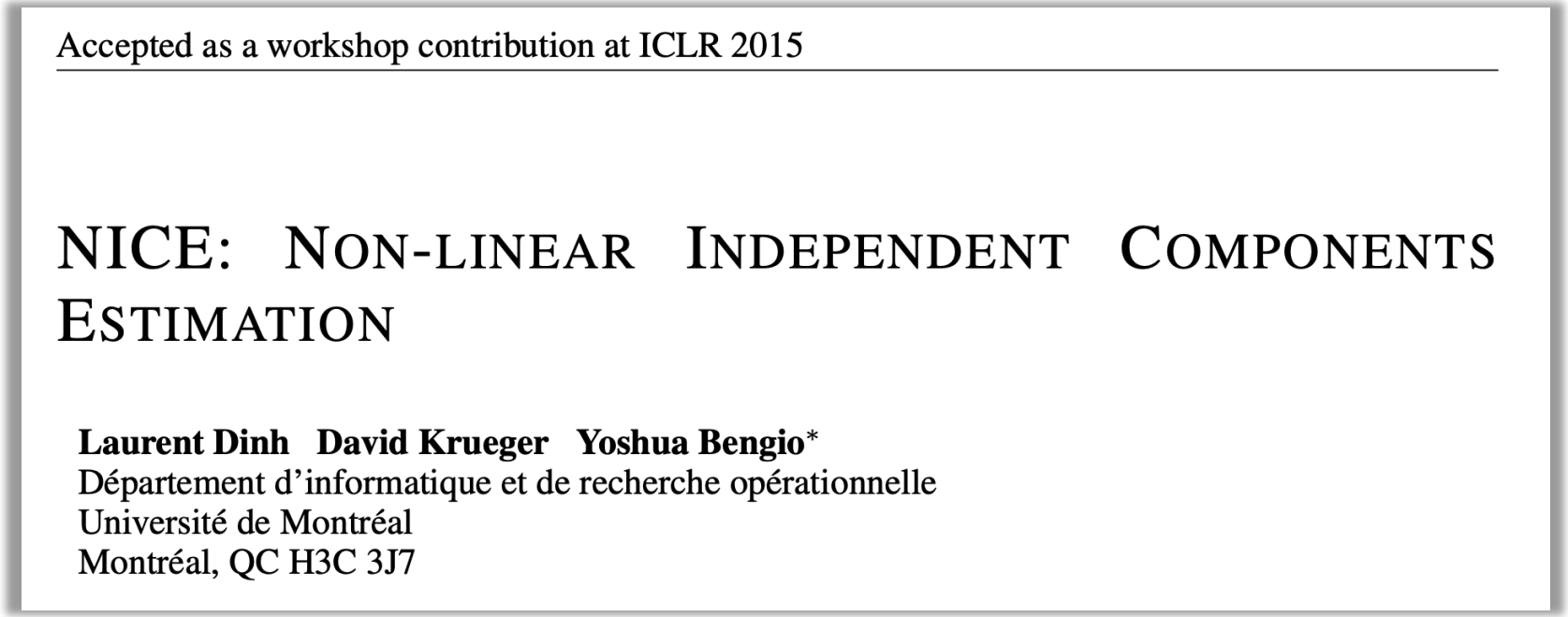
Laurent Dinh David Krueger Yoshua Bengio*
Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

- NICE is a flow-based model with the transformation f_θ is composed of
  - ✓ Additive coupling layers (加性耦合层)
  - ✓ Rescaling layers (缩放层)

Dinh et al., NICE: Non-linear Independent Components Estimation, ICLR (Workshop), 2015.

Flow Models with Triangular Jacobians

- **Non-linear Independent Components Estimation (NICE)**



Accepted as a workshop contribution at ICLR 2015

NICE: NON-LINEAR INDEPENDENT COMPONENTS ESTIMATION

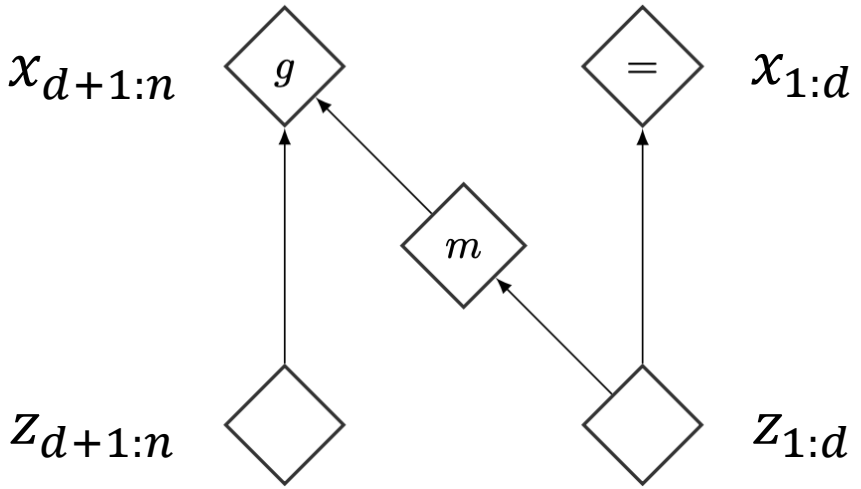
Laurent Dinh David Krueger Yoshua Bengio*
Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

- NICE is a flow-based model with the transformation f_θ is composed of
  - ✓ **Additive coupling layers (加性耦合层)**
  - ✓ Rescaling layers (缩放层)

Dinh et al., NICE: Non-linear Independent Components Estimation, ICLR (Workshop), 2015.

Flow Models with Triangular Jacobians

- **NICE: Additive Coupling Layers**
  - **Latent Partition:** The variable z is partitioned into two disjoint sets
    - ✓ $z_{1:d}$ and $z_{d+1:n}$, for any $1 \leq d \leq n$
  - **Forward Mapping $\boldsymbol{z \rightarrow x}$:**
    - ✓ "Carry-over"
      - $x_{1:d} = z_{1:d}$
    - ✓ **Additive Coupling**
      - $x_{d+1:n} = z_{d+1:n} + m_\theta(z_{1:d})$
      - $m_\theta(z_{1:d})$ is a neural network with parameter θ, d input units, and $n - d$ output units



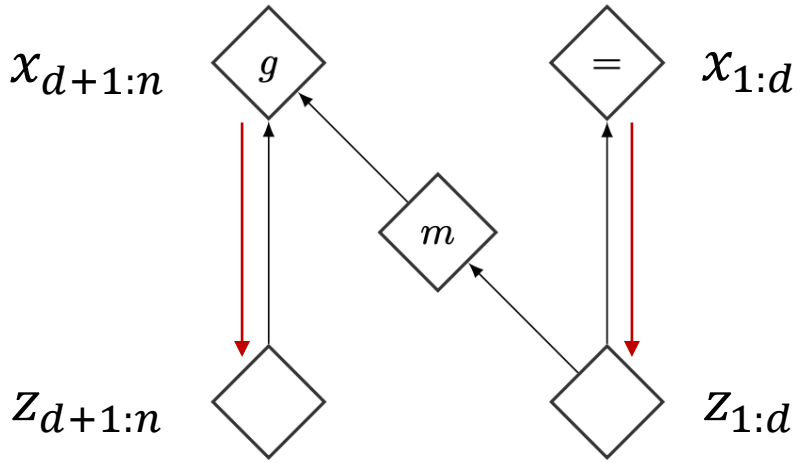
Flow Models with Triangular Jacobians

- **Is Additive Coupling Layers Invertible?**
  - **Forward Mapping $z \rightarrow x$:**
    - ✓ Identity Transformation: $x_{1:d} = z_{1:d}$
    - ✓ Additive Coupling: $x_{d+1:n} = z_{d+1:n} + m_\theta(z_{1:d})$
  - **Inverse Mapping $x \rightarrow z$:**
    - ✓ Defining $z \leftarrow f^{-1}(x)$
    - ✓ The first d dimensions (unchanged):

$$z_{1:d} = x_{1:d}$$

    - ✓ Other dimensions (shifted)

$$z_{d+1:n} = x_{d+1:n} - m_\theta(z_{1:d})$$



Flow Models with Triangular Jacobians

- **Is the Jacobian Tractable?**
  - **Forward Mapping $z \to x$:**
    - ✓ Identity Transformation: $x_{1:d} = z_{1:d}$
    - ✓ Additive Coupling: $x_{d+1:n} = z_{d+1:n} + m_\theta(z_{1:d})$
  - **Jacobian of Forward Mapping**

$$J = \frac{\partial \mathbf{x}}{\partial \mathbf{z}} = \begin{pmatrix} I_d & 0 \\ \frac{\partial \mathbf{x}_{d+1:n}}{\partial \mathbf{z}_{1:d}} & I_{n-d} \end{pmatrix}$$

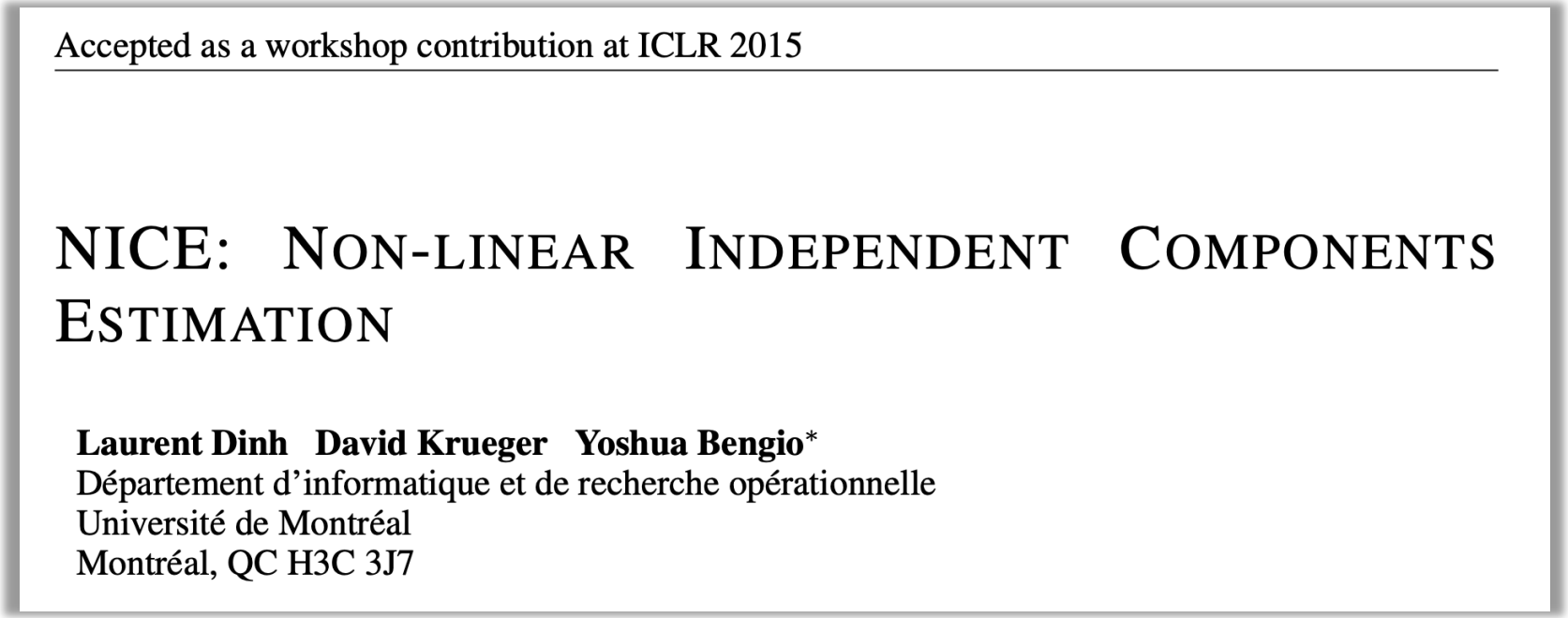
Triangular Jacobian Matrix!

$$\det(J) = 1$$

  - **Note:** ① Determinant is independent of m_θ; ② $f^{-1}(\cdot)$ can be computed **regardless of the invertibility m_θ**

Flow Models with Triangular Jacobians

- **Non-linear Independent Components Estimation (NICE)**



Accepted as a workshop contribution at ICLR 2015

NICE: NON-LINEAR INDEPENDENT COMPONENTS ESTIMATION

Laurent Dinh David Krueger Yoshua Bengio*
Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

- NICE is a flow-based model with the transformation f_θ is composed of
  - ✓ Additive coupling layers (加性耦合层)
  - ✓ **Rescaling layers (缩放层)**

Dinh et al., NICE: Non-linear Independent Components Estimation, ICLR (Workshop), 2015.

Flow Models with Triangular Jacobians

- **NICE: Re-Scaling Layers**
  - **Motivation:** Additive Coupling Layers are **volume-preserving** ($\det(J) = 1$)

    ➔ Cannot re-scale the overall density, thus **less expressive**
  - **Forward Mapping $z \to x$:**

$$x_i = s_i \cdot z_i$$

    where $s_i > 0$ is the scaling factor for the i-th dimension
  - **Inverse Mapping $z \to x$:**

$$z_i = x_i \,/\, s_i$$

  - **Jacobian Matrix:**

$$\boldsymbol{J} = \mathrm{diag}(s)$$

$$\det(J) = \prod_{i=1}^{n} s_i$$

Flow Models with Triangular Jacobians

- **NICE: Generation Results**



(a) Model trained on MNIST

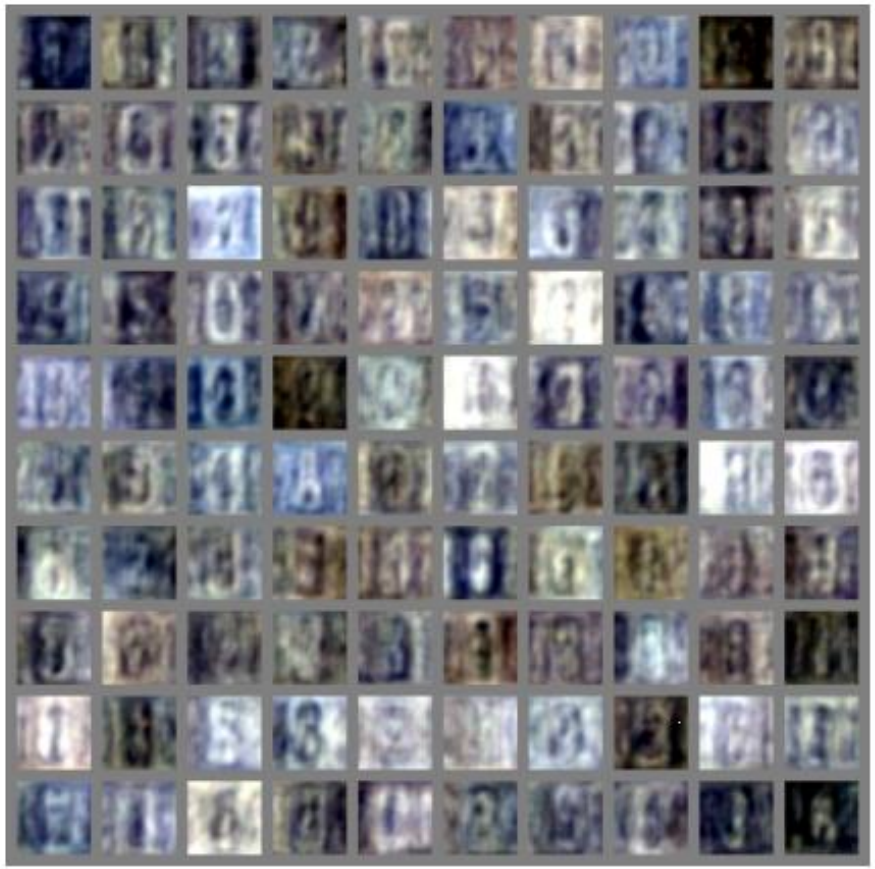


(b) Model trained on TFD

Dinh et al., NICE: Non-linear Independent Components Estimation, ICLR (Workshop), 2015.

Flow Models with Triangular Jacobians

- **NICE: Generation Results**



(c) Model trained on SVHN

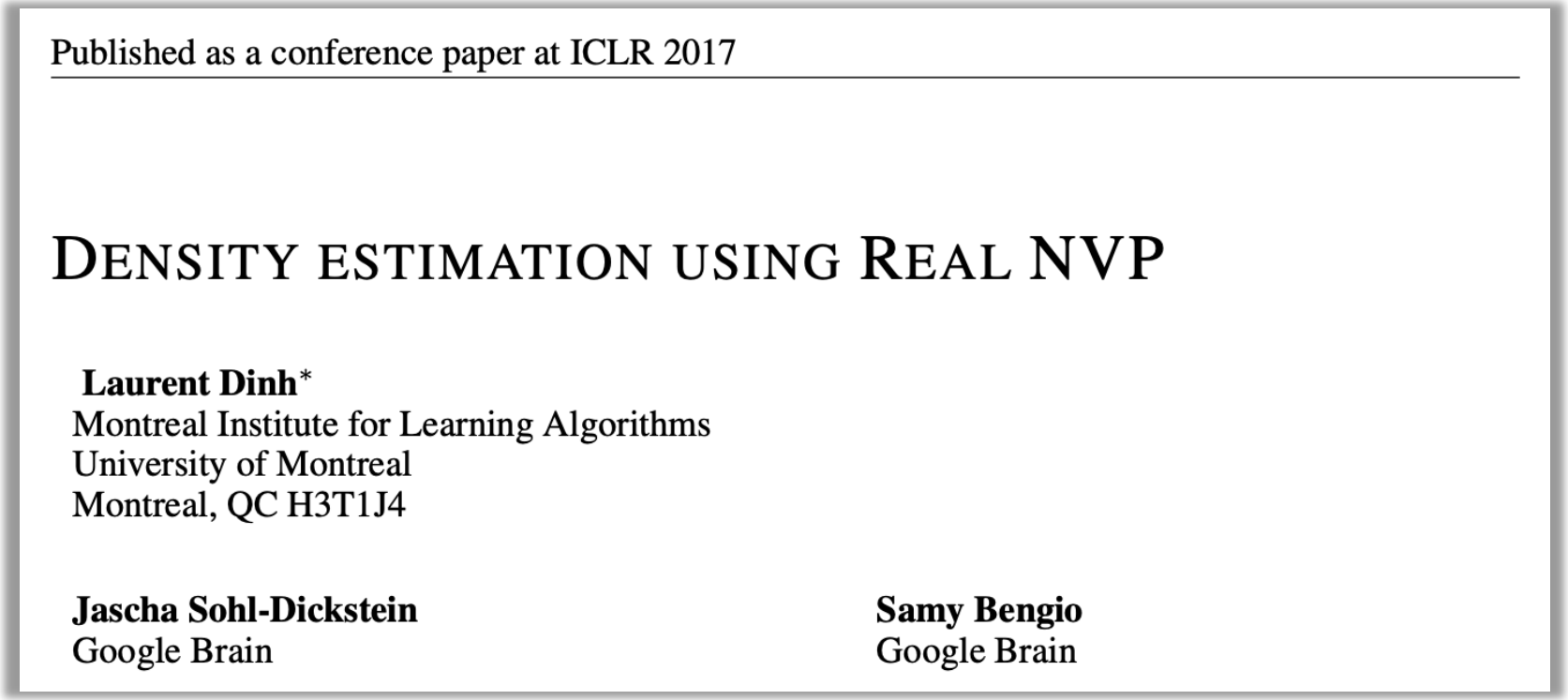


(d) Model trained on CIFAR-10

Dinh et al., NICE: Non-linear Independent Components Estimation, ICLR (Workshop), 2015.

Flow Models with Triangular Jacobians

- **Real-NVP**



Published as a conference paper at ICLR 2017

DENSITY ESTIMATION USING REAL NVP

Laurent Dinh*
Montreal Institute for Learning Algorithms
University of Montreal
Montreal, QC H3T1J4

Jascha Sohl-Dickstein
Google Brain

Samy Bengio
Google Brain

- **Real-NVP:** Real-valued **Non-Volume Preserving** Transformation
  - ✓ Extending the expressiveness of NICE

Dinh et al., DENSITY ESTIMATION USING REAL NVP, ICLR, 2017.

Flow Models with Triangular Jacobians

- **Real-NVP: Non-Volume Preserving Extension of NICE**
  - **Forward Mapping $z \to x$:**
    - ✓ Identity Transformation: $x_{1:d} = z_{1:d}$
    - ✓ **Affine Coupling**: $x_{d+1:n} = z_{d+1:n} \odot \exp\left(\alpha_\theta(z_{1:d})\right) + \mu_\theta(z_{1:d})$
      - $\alpha_\theta(\cdot)$ and $\mu_\theta(\cdot)$ are both neural networks with parameter θ, d input units, and $n - d$ output units
      - Containing both **shifting** and **scaling**, both are **data-dependent**
    - ✓ **Compare to NICE**
      - $x_{d+1:n} = z_{d+1:n} + m_\theta(z_{1:d})$
      - **Re-scaling layer is non data-dependent** $(x_i = s_i \cdot z_i;\ z_i = x_i \,/\, s_i)$

Flow Models with Triangular Jacobians

- **Real-NVP: Non-Volume Preserving Extension of NICE**
  - **Forward Mapping $z \rightarrow x$:**
    - ✓ Identity Transformation: $x_{1:d} = z_{1:d}$
    - ✓ **Affine Coupling**: $x_{d+1:n} = z_{d+1:n} \odot \exp\left(\alpha_\theta(z_{1:d})\right) + \mu_\theta(z_{1:d})$
  - **Inverse Mapping $x \rightarrow z$:**
    - ✓ $z_{1:d} = x_{1:d}$
    - ✓ $z_{d+1:n} = \left(x_{d+1:n} - \mu_\theta(z_{1:d})\right) \odot exp\left(\alpha_\theta(z_{1:d})\right)$

Flow Models with Triangular Jacobians

- **Real-NVP: Non-Volume Preserving Extension of NICE**
  - **Forward Mapping $z \to x$:**
    - ✓ Identity Transformation: $x_{1:d} = z_{1:d}$
    - ✓ **Affine Coupling**: $x_{d+1:n} = z_{d+1:n} \odot \exp\big(\alpha_\theta(z_{1:d})\big) + \mu_\theta(z_{1:d})$
  - **Jacobian Matrix and Determinant:**

$$J = \frac{\partial \mathbf{x}}{\partial \mathbf{z}} = \begin{pmatrix} I_d & 0 \\ \frac{\partial \mathbf{x}_{d+1:n}}{\partial \mathbf{z}_{1:d}} & \mathrm{diag}(\exp(\alpha_\theta(\mathbf{z}_{1:d}))) \end{pmatrix}$$

$$\det(J) = \prod_{i=d+1}^{n} \exp(\alpha_\theta(\mathbf{z}_{1:d})_i) = \exp\left(\sum_{i=d+1}^{n} \alpha_\theta(\mathbf{z}_{1:d})_i\right)$$

  - The overall transformation is non-volume preserving

Flow Models with Triangular Jacobians

- **NICE**
  - **Forward Transformation**

$$x_{1:d} = z_{1:d}$$

$$x_{d+1:n} = z_{d+1:n} + m_\theta(z_{1:d})$$

$$x_i = s_i \cdot z_i$$

  - **Jacobian and Determinant**

$$J = \mathrm{diag}(s)$$

$$\det(J) = \prod_{i=1}^{n} s_i$$

- **Real-NVP**
  - **Forward Transformation**

$$x_{1:d} = z_{1:d}$$

$$x_{d+1:n} = z_{d+1:n} \odot \exp\big(\alpha_\theta(z_{1:d})\big) + \mu_\theta(z_{1:d})$$

  - **Jacobian and Determinant**

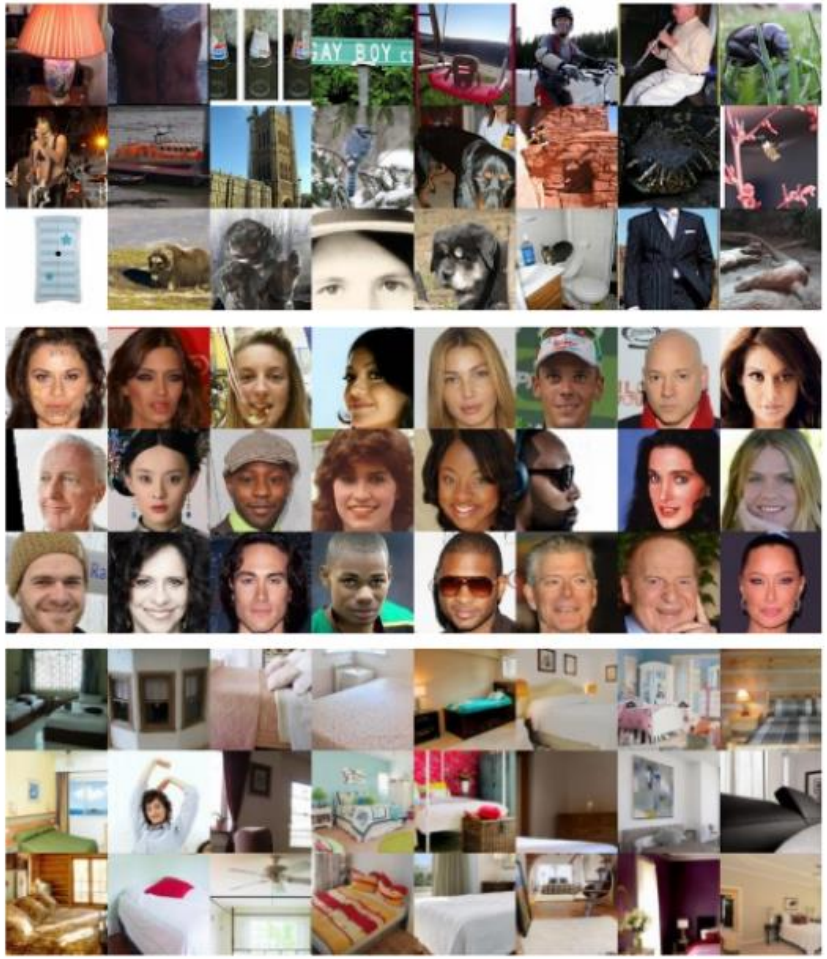
$$J = \frac{\partial \mathbf{x}}{\partial \mathbf{z}} = \begin{pmatrix} I_d & 0 \\ \frac{\partial \mathbf{x}_{d+1:n}}{\partial \mathbf{z}_{1:d}} & \mathrm{diag}(\exp(\alpha_\theta(\mathbf{z}_{1:d}))) \end{pmatrix}$$

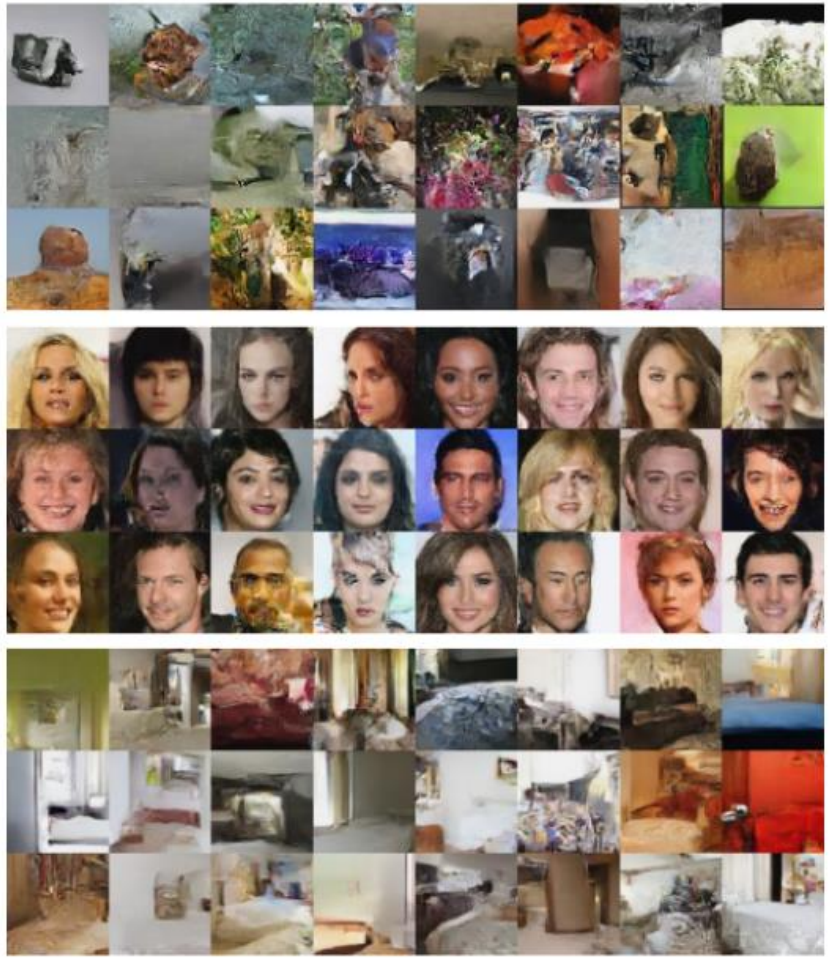
$$\det(J) = \prod_{i=d+1}^{n} \exp(\alpha_\theta(\mathbf{z}_{1:d})_i) = \exp\left(\sum_{i=d+1}^{n} \alpha_\theta(\mathbf{z}_{1:d})_i\right)$$

Flow Models with Triangular Jacobians

- **Real-NVP: Generation Results**

Original Images





Generated Samples

Dinh et al., DENSITY ESTIMATION USING REAL NVP, ICLR, 2017.

Flow Models with Triangular Jacobians

- **Real-NVP: Interpolation Results**

$z^{(1)} = f^{-1}(x^{(1)})$





$z^{(2)} = f^{-1}(x^{(2)})$

$z^{(3)} = f^{-1}(x^{(3)})$





$z^{(4)} = f^{-1}(x^{(4)})$

- Interpolating the latent $\boldsymbol{z}$ within a parameterized manifold via:

$$z = \cos\phi\left(z^{(1)}\cos\phi' + z^{(2)}\sin\phi'\right) + \sin\phi\left(z^{(3)}\cos\phi' + z^{(4)}\sin\phi'\right)$$

Dinh et al., DENSITY ESTIMATION USING REAL NVP, ICLR, 2017.



Lecture Outline

- **Flow Models with Triangular Jacobians**

- **Autoregressive Flow Models**
  - ✓ **MAF Models**
  - ✓ **IAF Models**

- **Probability Distillation and Parallel WaveNet**

Autoregressive Flow Models

- **Autoregressive Model as Flow Models**
  - Consider an autoregressive model

$$p(x) = \prod_{i=1}^{n} p(x_i \mid x_{<i})$$

where the conditionals are Gaussians (**Gaussian AR Model**)

$$p(x_i \mid x_{<i}) = \mathcal{N}\left(\mu_i(x_1, \ldots, x_{i-1}), \exp\left(\alpha_i(x_1, \ldots, x_{i-1})\right)\right)$$

  - **Note:**
    - ✓ $\mu_i(\cdot)$ and $\alpha_i(\cdot)$ are neural networks for $i > 1$
    - ✓ μ_1 and α_1 are constants (prior distribution)

Autoregressive Flow Models

- **Autoregressive Model as Flow Models**

$$p(x) = \prod_{i=1}^{n} p(x_i \mid x_{<i})$$

$$p(x_i \mid x_{<i}) = \mathcal{N}\left(\mu_i(x_1, \ldots, x_{i-1}), \exp\left(\alpha_i(x_1, \ldots, x_{i-1})\right)\right)$$



How to sample from such a distribution?

Autoregressive Flow Models

- **Autoregressive Model as Flow Models**

$$p(x) = \prod_{i=1}^{n} p(x_i \mid x_{<i})$$

$$p(x_i \mid x_{<i}) = \mathcal{N}\left(\mu_i(x_1, \ldots, x_{i-1}), \exp\left(\alpha_i(x_1, \ldots, x_{i-1})\right)\right)$$

  - **An alternative sampler:**

    ① Sample $z_i \sim \mathcal{N}(0,1)$ for $i = 1, \ldots, n$

    ② Let $x_1 = \exp(\alpha_1)\, z_1 + \mu_1$. Compute $\mu_2(x_1)$, $\alpha_2(x_1)$

    ③ Let $x_2 = \exp(\alpha_2)\, z_2 + \mu_2$. Compute $\mu_3(x_1, x_2)$, $\alpha_3(x_1, x_2)$

    ④ Let $x_3 = \exp(\alpha_3)\, z_3 + \mu_3$. Compute …

Autoregressive Flow Models

- **Autoregressive Model as Flow Models**
  - **An Alternative Sampler:**

    ① Sample $z_i \sim \mathcal{N}(0,1)$ for $i = 1, \dots, n$

    ② Let $x_1 = \exp(\alpha_1)\, z_1 + \mu_1$. Compute $\mu_2(x_1)$, $\alpha_2(x_1)$

    ③ Let $x_2 = \exp(\alpha_2)\, z_2 + \mu_2$. Compute $\mu_3(x_1, x_2)$, $\alpha_3(x_1, x_2)$

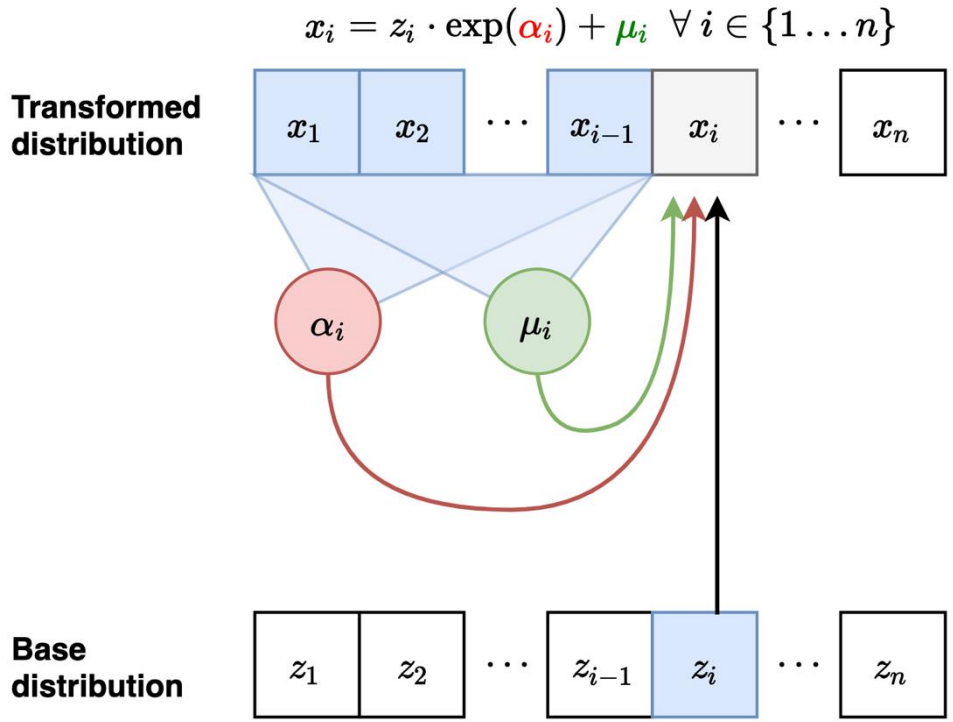
    ④ Let $x_3 = \exp(\alpha_3)\, z_3 + \mu_3$. Compute …

  - **Flow Model Interpretation:** Sample $(z_1, z_2, \dots, z_n)$ from standard Gaussian to generated data $(x_1, x_2, \dots, x_n)$ via **invertible mappings**

    ➔ Can be **interpreted** as a normalizing flow model !

Autoregressive Flow Models

- **Masked Autoregressive Flow (MAF)**
  - **Definition:** The transformation $f: Z \rightarrow X$ in MAF implements

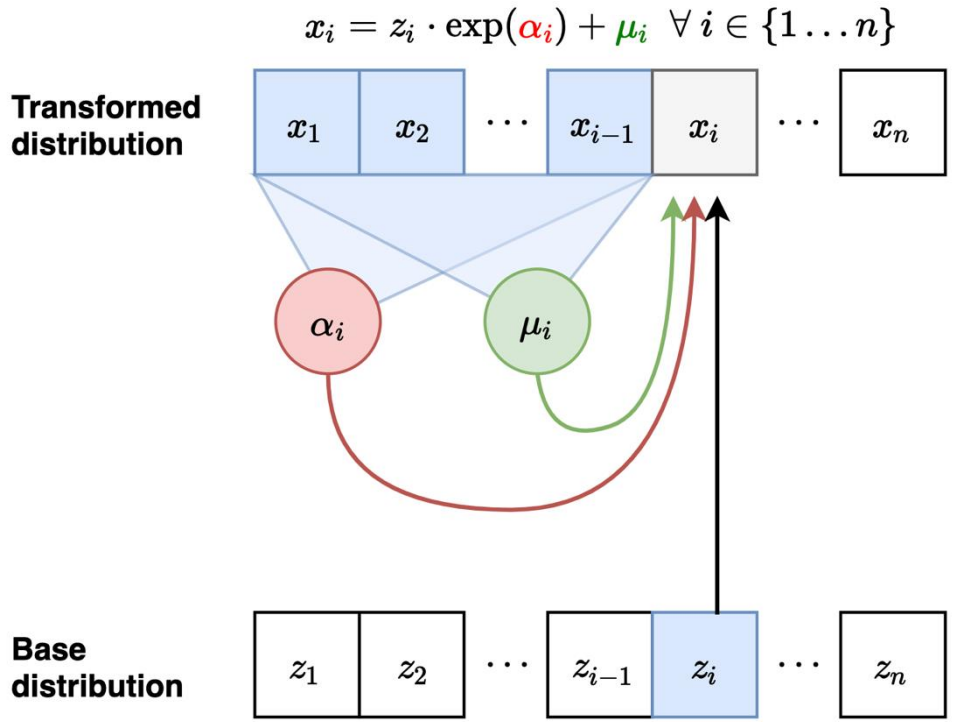


$$p(x_i \mid x_{<i}) = \mathcal{N}\left(\mu_i(x_1, \ldots, x_{i-1}), \exp\left(\alpha_i(x_1, \ldots, x_{i-1})\right)\right)$$

Papamakarios, G., Pavlakou, T., & Murray, I. Masked Autoregressive Flow for Density Estimation. NeurIPS, 2017.

Autoregressive Flow Models

- **Masked Autoregressive Flow (MAF)**
  - **Definition:** The transformation $f: Z \rightarrow X$ in MAF implements



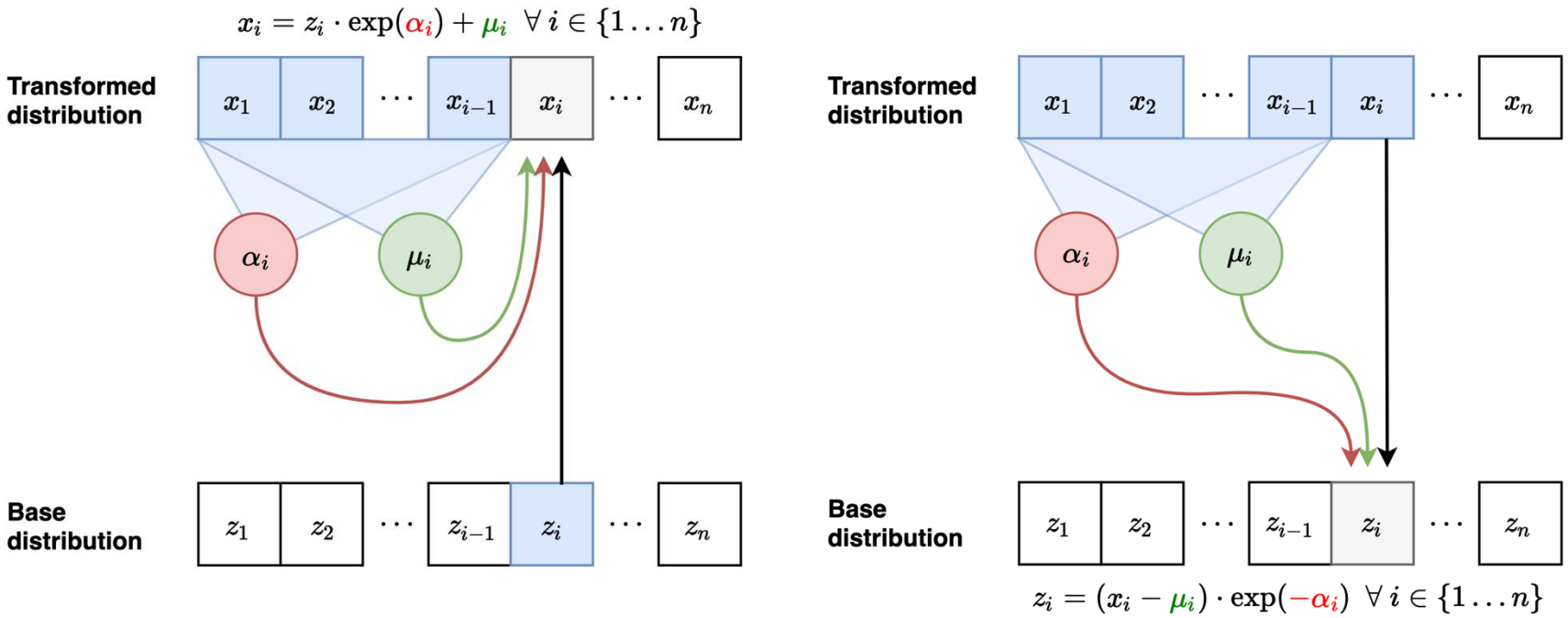
  - Forward mapping (Sampling) is **sequential and slow (AR)** ➔ $O(n)$ time

Papamakarios, G., Pavlakou, T., & Murray, I. Masked Autoregressive Flow for Density Estimation. NeurIPS, 2017.

Autoregressive Flow Models

- **Masked Autoregressive Flow (MAF)**

Forward Mapping $z \rightarrow x$



Inverse Mapping $x \rightarrow z$

- **Forward Mapping $z \rightarrow x$:**

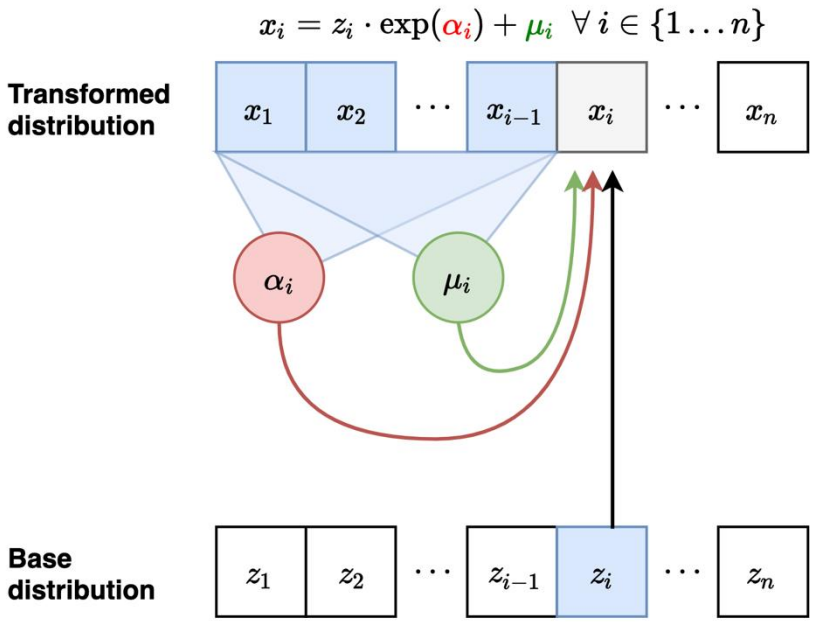
① Sample $z_i \sim \mathcal{N}(0,1)$ for $i = 1, \dots, n$

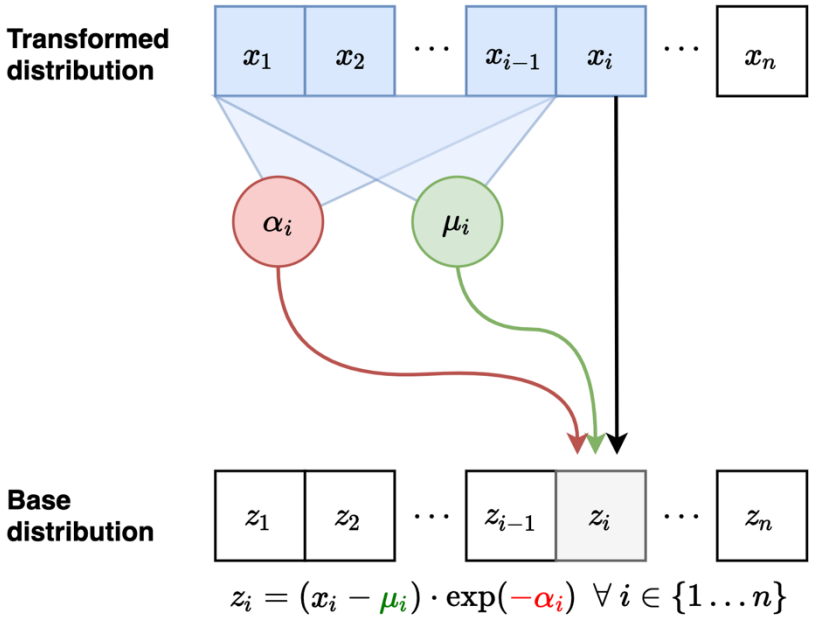
② Let $x_i = \exp(\alpha_i)\, z_i + \mu_i$. Compute $\mu_i(x_{<i})$, $\alpha_i(x_{<i})$

Autoregressive Flow Models

- **Masked Autoregressive Flow (MAF)**

Forward Mapping
$z \rightarrow x$





Inverse Mapping
$x \rightarrow z$

- **Inverse Mapping $x \rightarrow z$:**
  - ① Compute all μ_i, α_i ➔ done **in parallel via masked network**
  - ② Compute all $z_i = \frac{x_i - \mu_i(x_{<i})}{\exp(\alpha_i(x_{<i}))}$ (**in parallel**)

Autoregressive Flow Models

- **Masked Autoregressive Flow (MAF)**
  - ➢ **Inverse Mapping $x \to z$:**
    - ✓ Compute all μ_i, α_i ➔ done **in parallel via masked network**
    - ✓ Compute all $z_i = \frac{x_i - \mu_i(x_{<i})}{\exp(\alpha_i(x_{<i}))}$ (**in parallel**)
  - ➢ **Properties of MAF:**
    - ① Forward mapping is autoregressive ➔ Sampling is slow
    - ② Inverse mapping is parallelizable ➔ **Likelihood evaluation is efficient**

$$p_X(x;\theta) = p_Z\left(f_\theta^{-1}(x)\right)\left|det\left(\frac{\partial f_\theta^{-1}(x)}{\partial x}\right)\right|$$

    - ③ Jacobian is triangular (AR-ness) ➔ **Determinant computation is easy**



Lecture Outline

- **Flow Models with Triangular Jacobians**

- **Autoregressive Flow Models**
  - ✓ **MAF Models**
  - ✓ **IAF Models**

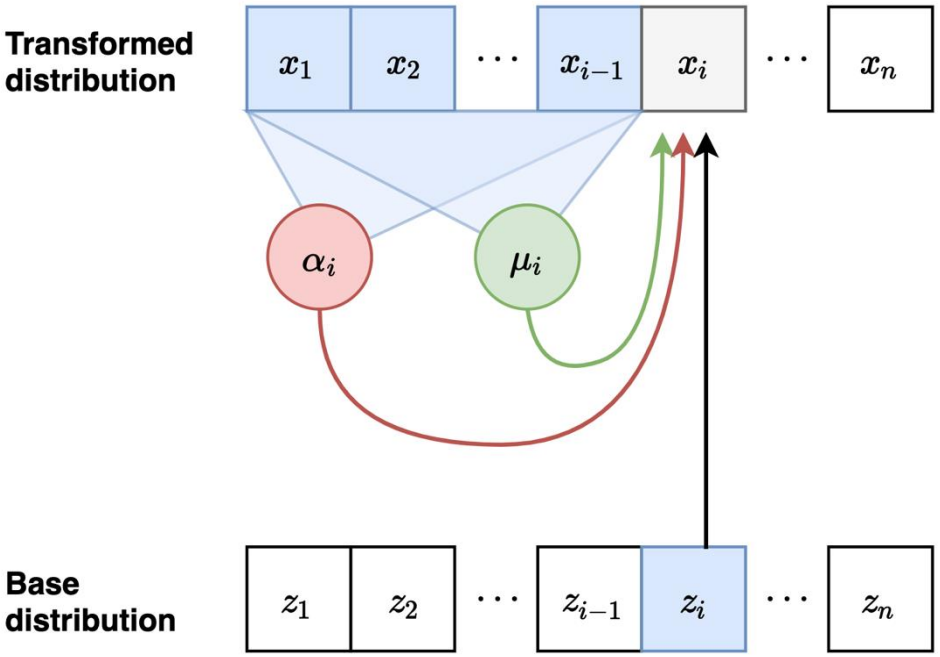
- **Probability Distillation and Parallel WaveNet**

Autoregressive Flow Models

- **Inverse Autoregressive Flow (IAF)**

$$p_X(x) = \prod_{i=1}^{n} p_X\left(x_i \mid \boldsymbol{x_{<i}}\right)$$

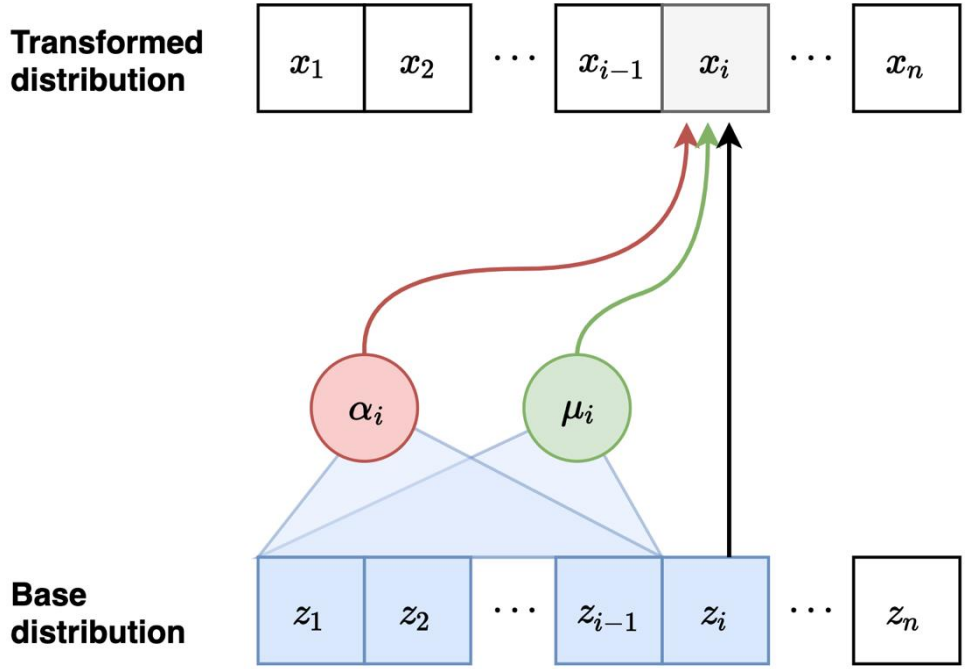
$$x_i = z_i \cdot \exp(\alpha_i) + \mu_i \quad \forall\, i \in \{1 \dots n\}$$



MAF: Autoregressive on X

$$p_X(x) = \prod_{i=1}^{n} p_X\left(x_i \mid \boldsymbol{z_{\leq i}}\right)$$

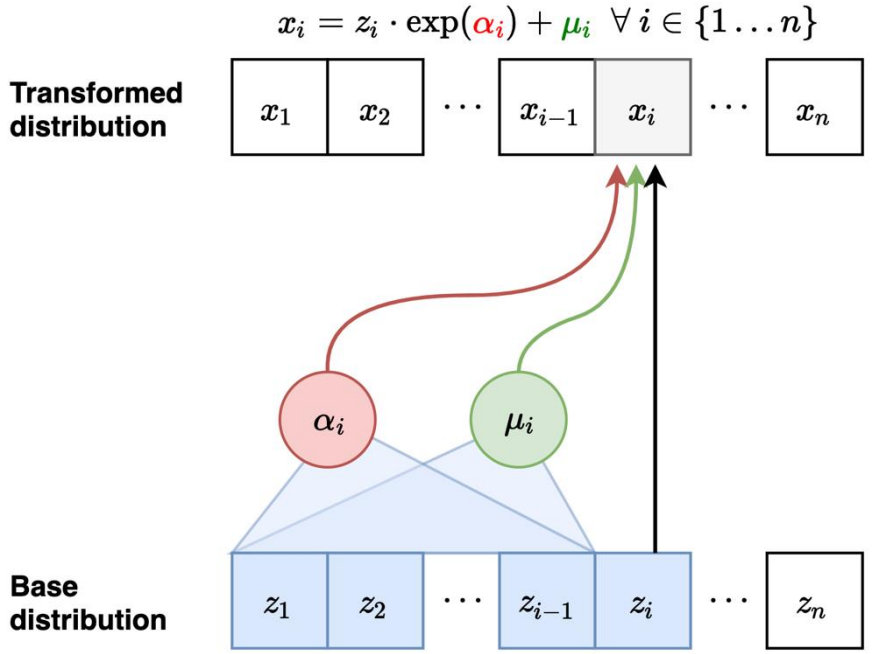
$$x_i = z_i \cdot \exp(\alpha_i) + \mu_i \quad \forall\, i \in \{1 \dots n\}$$



IAF: Autoregressive on Z

Autoregressive Flow Models

- **Inverse Autoregressive Flow (IAF)**



IAF Models

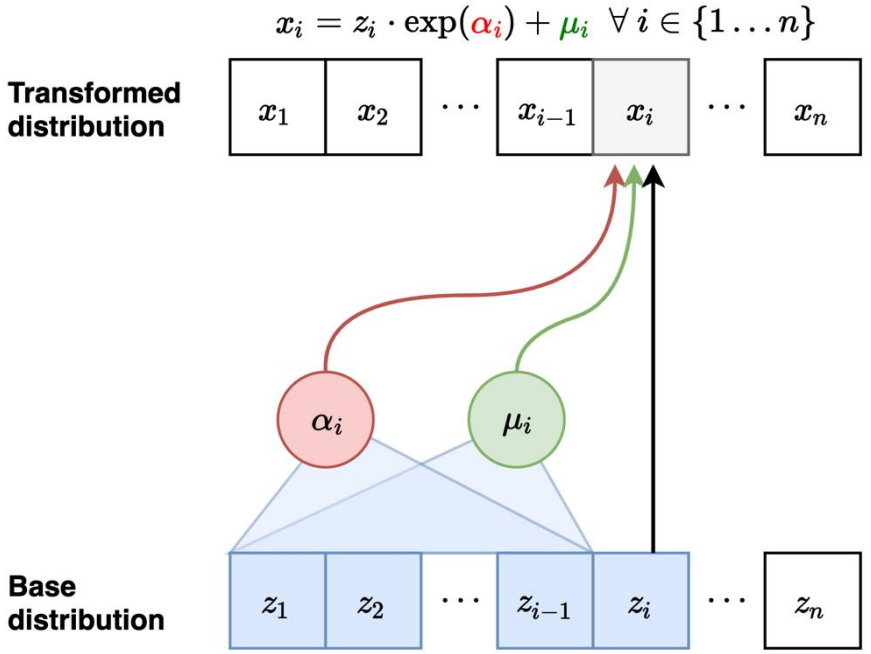
➢ **Forward Mapping $z \to x$:**

✓ Sample $z_i \sim \mathcal{N}(0,1)$ and compute all $\mu_i(z_{<i})$, $\alpha_i(z_{<i})$ ➔ **in parallel (fast)**

✓ Let $x_i = \exp(\alpha_i) z_i + \mu_i$ ➔ **in parallel (fast)**

Autoregressive Flow Models

- **Inverse Autoregressive Flow (IAF)**



IAF Models

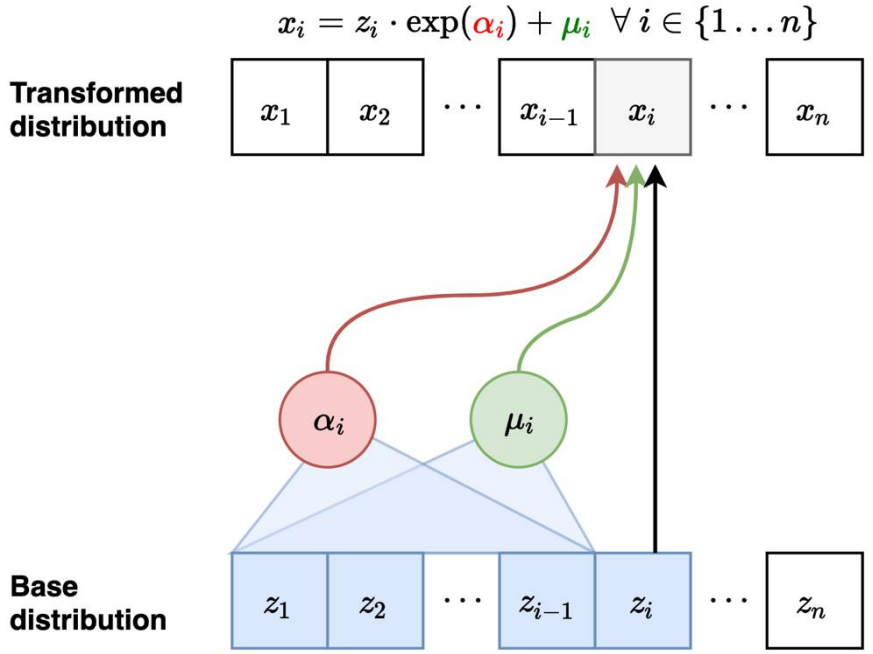
➢ **Inverse Mapping $x \rightarrow z$:** Can only be compute **sequentially (slow)**

$$z_i = \frac{x_i - \mu_i(z_{<i})}{\exp\left(\alpha_i(z_{<i})\right)}$$

z_i is dependent on $z_{<i}$ given x_i

Autoregressive Flow Models

- **Inverse Autoregressive Flow (IAF)**



IAF Models

➢ **Inverse Mapping $x \rightarrow z$:** Can only be compute **sequentially (slow)**

✓ Let $z_1 = (x_1 - \mu_1)/\exp(\alpha_1)$, then compute $\mu_2(z_1)$, $\alpha_2(z_1)$

✓ Let $z_2 = (x_2 - \mu_2)/\exp(\alpha_2)$, then compute $\mu_3(z_1, z_2)$, $\alpha_3(z_1, z_2)$



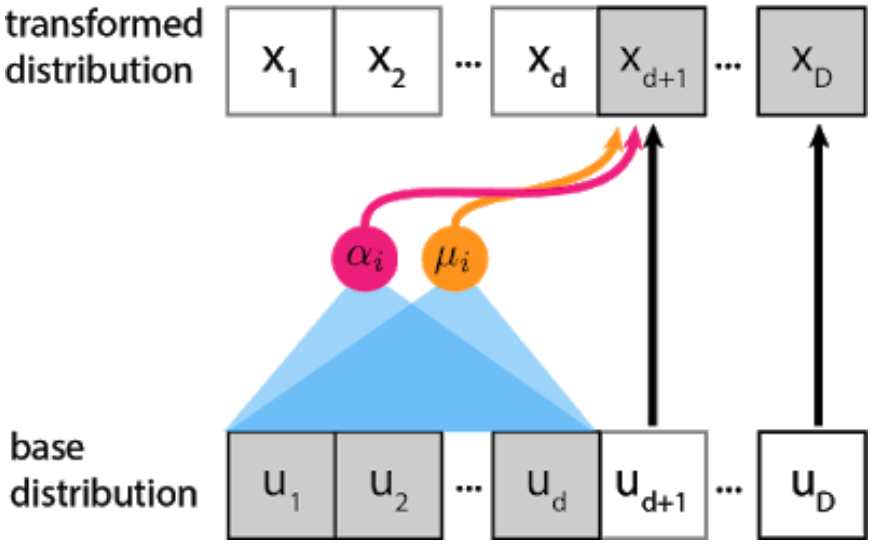
Autoregressive Flow Models

- **Inverse Autoregressive Flow (IAF)**
  - **Forward Mapping $z \to x$: Parallel and Fast**
    - ✓ Sample $z_i \sim \mathcal{N}(0,1)$ and compute all $\mu_i(z_{<i})$, $\alpha(z_{<i})$
    - ✓ Let $x_i = \exp(\alpha_i) z_i + \mu_i$
  - **Inverse Mapping $x \to z$: Sequentially and Slow**
    - ✓ Let $z_1 = (x_1 - \mu_1)/\exp(\alpha_1)$, then compute $\mu_2(z_1)$, $\alpha_2(z_1)$
    - ✓ Let $z_2 = (x_2 - \mu_2)/\exp(\alpha_2)$, then compute $\mu_3(z_1, z_2)$, $\alpha_3(z_1, z_2)$
  - **Important Note:**
    - ✓ Still fast to evaluate likelihood of a **generated** point
      - ➔ By **caching** $z_1, \ldots, z_n$. Not true for an arbitrary data point

Autoregressive Flow Models

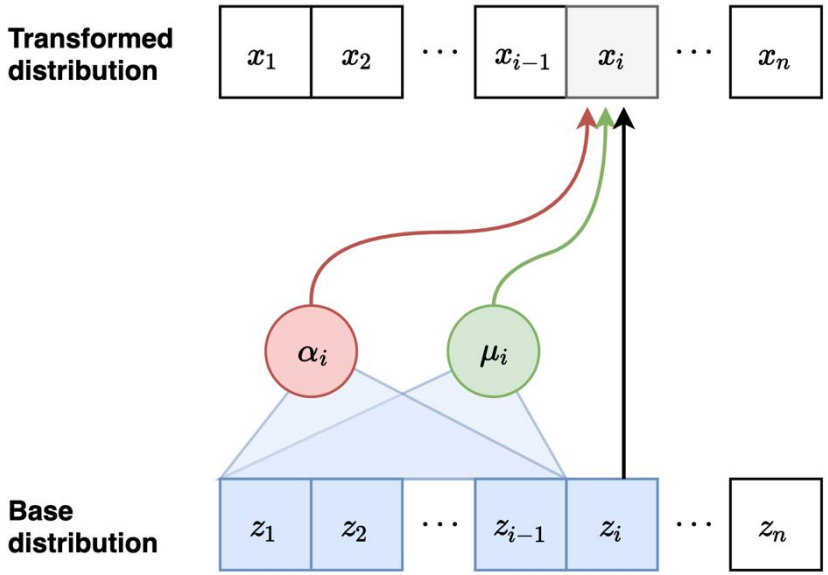
- **NICE and Real NVP as IAF**
  - ➢ NICE and Real NVP are special cases of the IAF framework

$$\begin{cases} x_{1:d} = u_{1:d}, \\ x_{d+1:D} = u_{d+1:D} \odot \exp\left(\alpha(u_{1:d})\right) + \mu(u_{1:d}) \end{cases}$$



IAF Models

$$x_i = z_i \cdot \exp(\alpha_i) + \mu_i \quad \forall\, i \in \{1 \dots n\}$$



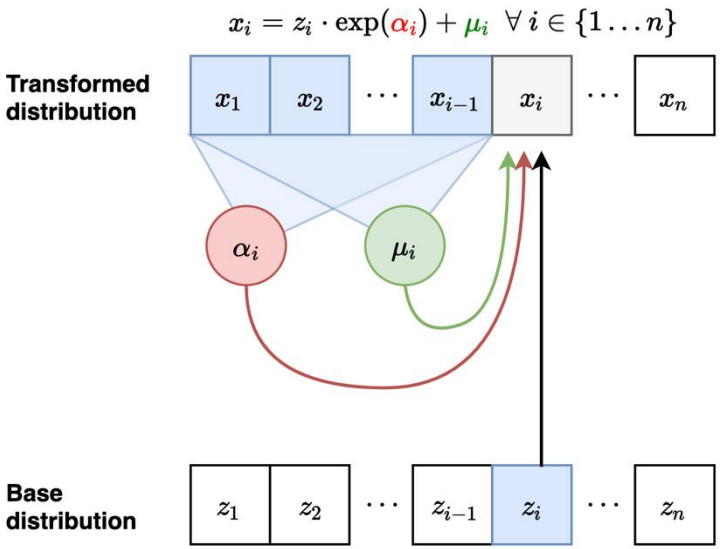
  - ✓ Shift and scale factors can be computed in a **single pass** (**non-AR**)

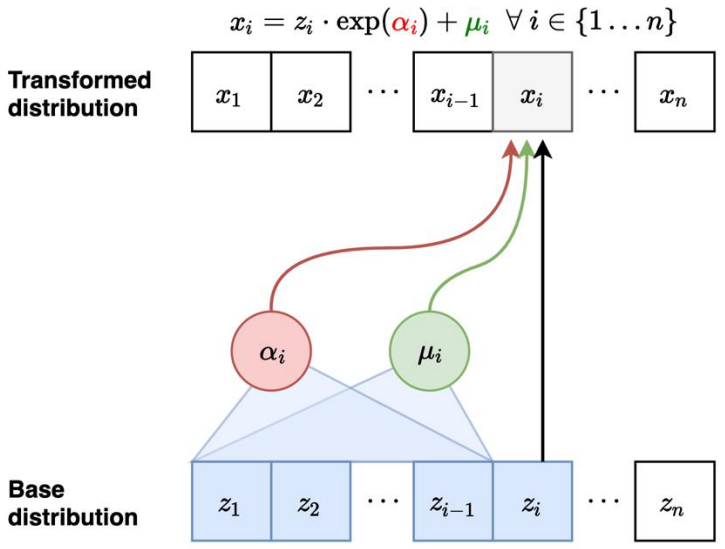
https://blog.evjang.com/2018/01/nf2.html

Autoregressive Flow Models

- **Discussion: IAF vs. MAF**
  - ➢ Both IAF and MAF are expressive models, with different trade-offs

MAF Models





IAF Models

Fast likelihood, **slow sampling**

➔ MLE / Density Estimation



Fast sampling, **slow likelihood**

➔ Real-time generation

➢ **Question:** **Can we get the best of both worlds?**



Lecture Outline

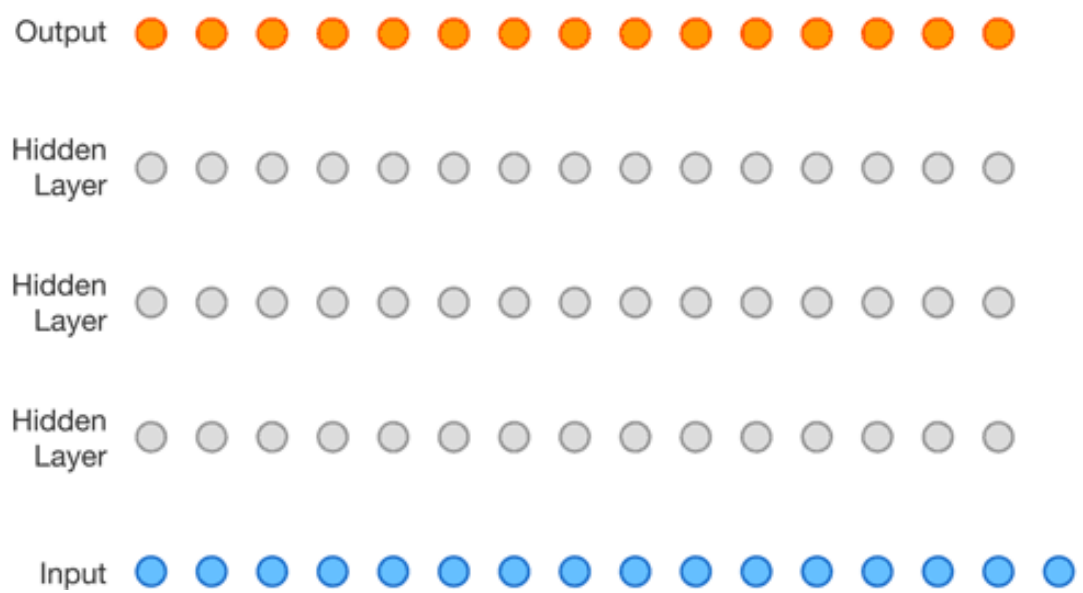
- **Flow Models with Triangular Jacobians**

- **Autoregressive Flow Models**
  - ✓ **MAF Models**
  - ✓ **IAF Models**

- **Probability Distillation and Parallel WaveNet**

Probability Distillation and Parallel WaveNet

- **Recap on WaveNet**
  - ➢ Audio generation model with casual and dilated 1-D convolution



✓ **Generating / Sampling from this autoregressive model is slow**

→ How to improve the efficiency using MAF and IAF?

van den Oord, et al. “WaveNet: A Generative Model for Raw Audio”, 2016

Probability Distillation and Parallel WaveNet

- **Model Distillation (蒸馏)**
  - **Probability Density Distillation:** A student distribution p_s is trained to minimize the KL divergence between the student (s) and the teacher (t)

$$D_{KL}(p_s \parallel p_t) = \mathbb{E}_{x\sim p_s}[\log p_s(x) - \log p_t(x)]$$

  - Evaluating and optimizing this objective (via **Monte Carlo**) requires
    ① Samples x from the student model
    ② Likelihood of x assigned by student model
    ③ Likelihood of x assigned by teacher model
  - **Question:** How to assign MAF and IAF to student and teacher models?

Probability Distillation and Parallel WaveNet

- **From WaveNet to Parallel WaveNet**

$$D_{KL}(p_s \parallel p_t) = \mathbb{E}_{x \sim p_s}[\log p_s(x) - \log p_t(x)]$$

- **Teacher Model:** Powerful model to be distilled
  - ✓ Fast likelihood evaluation
  - ✓ Easily trained using MLE
  - ✓ **Slow sampling → Not an issue for teacher model!**

MAF Models

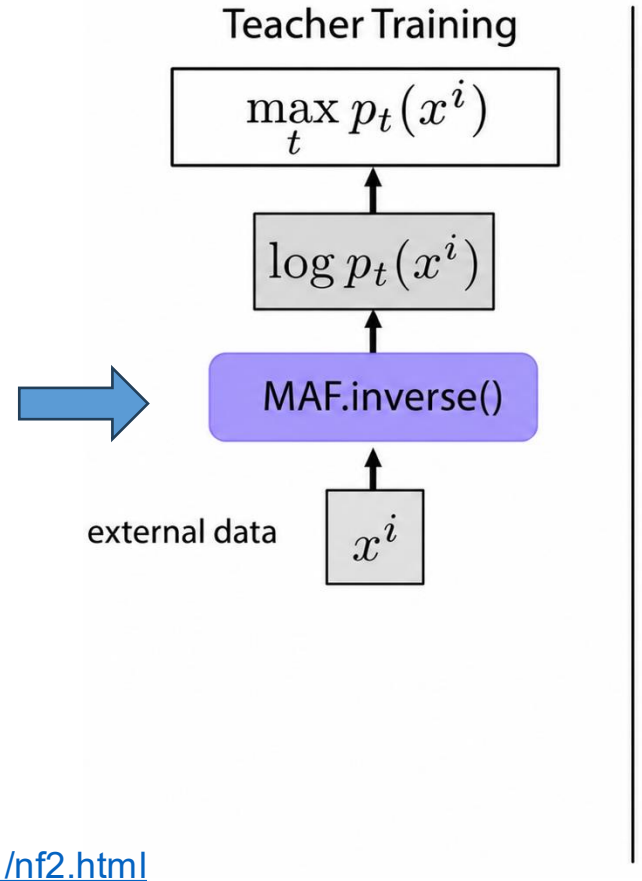
- **Student Model:** Target model to be trained
  - ✓ Fast sampling
  - ✓ Slow density (likelihood) estimation
  - → **But efficient for generated samples (via caching) !**

IAF Models

Probability Distillation and Parallel WaveNet

- **Parallel WaveNet**
  - **Parallel WaveNet:** Two-part training with a teacher and student model

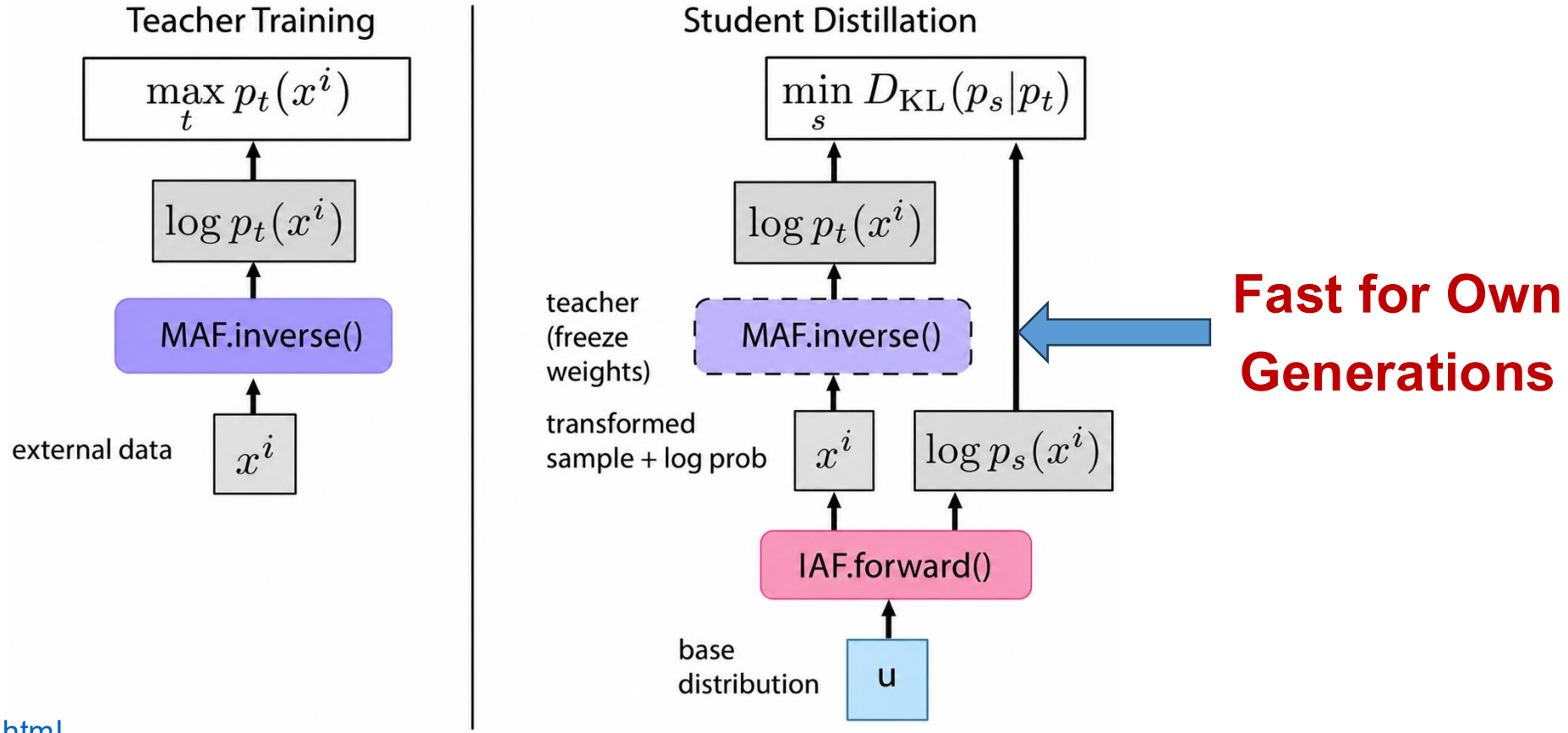
Fast Likelihood Estimation



- https://blog.evjang.com/2018/01/nf2.html
- van den Oord et al., Parallel WaveNet: Fast High-Fidelity Speech Synthesis, ICML, 2018.

Probability Distillation and Parallel WaveNet

- **Parallel WaveNet**
  - **Parallel WaveNet:** Two-part training with a teacher and student model



- https://blog.evjang.com/2018/01/nf2.html
- van den Oord et al., Parallel WaveNet: Fast High-Fidelity Speech Synthesis, ICML, 2018.

Probability Distillation and Parallel WaveNet

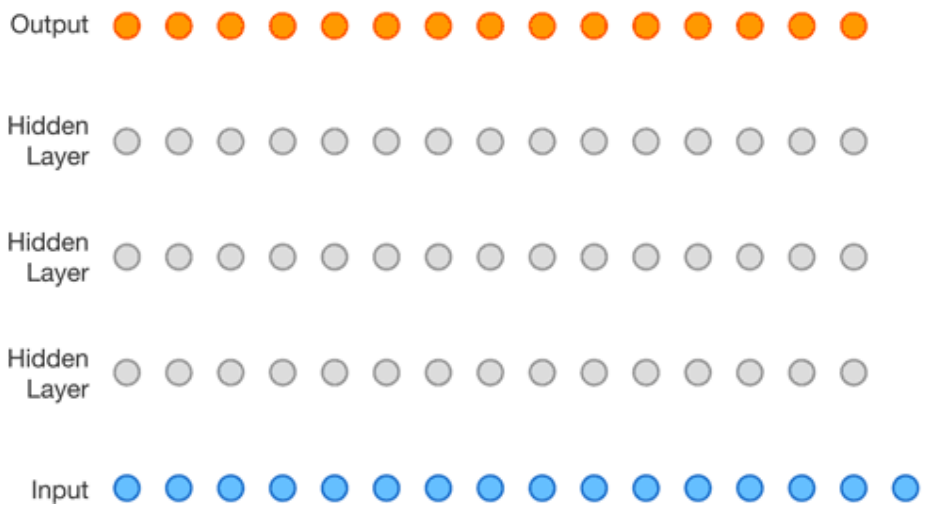
- **Parallel WaveNet**
  - **Parallel WaveNet:** Two-part training with a teacher and student model

Output x Generated
in parallel

Random White
Noise z

Probability Distillation and Parallel WaveNet

- **Parallel WaveNet**
  - **Parallel WaveNet:** Two-part training with a teacher and student model



Original WavetNet

Parallel WavetNet



Lecture Outline

- **Flow Models with Triangular Jacobians**

- **Autoregressive Flow Models**
  - ✓ **MAF Models**
  - ✓ **IAF Models**

- **Probability Distillation and Parallel WaveNet**

Summary of Normalizing Flow Models

- **Key Motivation**
  - Transform simple distributions p_Z into more complex ones p_X, using the invertible function f_θ with change of variables formula
- **Pros**
  - Exact marginal likelihood $p(x)$ is tractable to compute and optimize
  - Exact posterior inference $p(z \mid x)$ is tractable due to invertible transformation
- **Cons**
  - x and z are required to be continuous and of the same dimension
  - Post strict constraints on the design of neural network structure for f_θ

Summary of Normalizing Flow Models

- **Strategies for Flow Construction**
  - **Simple Bijection Composition**

$$z_M := f_\theta^M \circ \cdots \circ f_\theta^1(z_0) = f_\theta^M\left(f_\theta^{M-1}\left(\cdots\left(f_\theta^1(z_0)\right)\right)\right) \triangleq f_\theta(z_0)$$

  - **Triangular Jacobian Construction**
    - ✓ NICE, Real-NVP
  - **Autoregressive Implementation**:
    - Use autoregressive structure to build invertible flows
    - Trade-offs between forward and inverse speed

Supplementary Material

- **Stanford CS 236 Lecture 8**
  - https://www.youtube.com/watch?v=qgTvgBCOyn8&list=PLoROMvodv4rPOWA-omMM6STXaWW4FvJT8&index=8
- **Cornell CS 6785 Lecture 8**
  - https://www.youtube.com/watch?v=lEx8ZgeN8kQ&list=PL2UML_KCiC0UPzjW9BjO-lW6dqliu9O4B&index=8
- **Deep Generative Models**
  - Chapter 4 Flow-based Models

Supplementary Material

- **Reading List**
  - Dinh, Krueger & Bengio, NICE: Non-linear Independent Components Estimation, arXiv 2014.
  - Dinh, Sohl-Dickstein & Bengio, Density Estimation using Real NVP, ICLR, 2017.
  - Kingma et al., Improving Variational Inference with Inverse Autoregressive Flow, NeurIPS, 2016.
  - Papamakarios, Pavlakou & Murray, Masked Autoregressive Flow for Density Estimation, NeurIPS, 2017.
  - Kingma & Dhariwal, Glow: Generative Flow with Invertible 1×1 Convolutions, NeurIPS, 2018.
  - Grathwohl et al., FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models, ICLR, 2019.
  - van den Oord et al., Parallel WaveNet: Fast High-Fidelity Speech Synthesis, ICML, 2018.
  - Ping et al., ClariNet: Parallel Wave Generation in End-to-End Text-to-Speech, ICLR, 2019.